

Memo

To

Delft3D users

Date

2026-06-18

Our reference

001

Number of pages

3

Contact person

Johan Reyns

Direct line

E-mail

johan.reyns@deltares.nl

Subject

Slurm MPI Orchestration for MorMerge

1 Purpose

This note documents the Slurm orchestration path added to `mormerge.tcl` for MorMerge runs that use MPI-partitioned models. The implementation keeps the existing local, SGE, and Torque behavior intact, but removes the previous Slurm restriction that only allowed `nodes = 1`.

The Slurm path now uses one Slurm allocation for the complete MorMerge run. Within that allocation, the Tcl driver expands the allocated node list, assigns deterministic node slices to each condition, starts the serial MorMerge helper processes on the batch/control node, and starts each DIMR condition through an explicit `srun` MPI job step.

2 User-facing configuration

The existing `nodes`, `queueSystem`, `queue`, and `timeLimit` keys are still used. The Slurm path adds the following optional keys to the `.mm` file:

slurmControlNodes Nodes reserved before condition node slices. Default: 0.

slurmNodesPerCondition Nodes assigned to each condition. Default: 0 for automatic equal partitioning.

slurmTasksPerNode MPI ranks per node for DIMR condition steps. Default: 1.

slurmCpusPerTask CPU threads per MPI rank. Default: 1.

slurmMPI Optional `srun --mpi` value, for example `pmix` or `pmi2`. Use `none` to omit.

slurmModuleCommand Module command emitted in the generated `sbatch` script. Default: `module load intelmpi/2021.9.0`. Use `none` to omit.

Example for two MorMerge conditions, each running DIMR on two nodes with 32 MPI ranks per node, plus one control node:

```
queueSystem      = slurm
queue            = normal
timeLimit        = 02:00:00
nodes            = 5

dimrexedir       = /opt/delft3dfm/lnx64/bin
```

```
dimrargs                = dimr_config.xml
mormergeexedir           = /opt/delft3dfm/lnx64/bin

slurmControlNodes        = 1
slurmNodesPerCondition   = 2
slurmTasksPerNode        = 32
slurmCpusPerTask         = 1
slurmMPI                 = pmix
slurmModuleCommand       = module load intelmpi/2021.9.0

condition:weight         = storm_a : 0.5
condition:weight         = storm_b : 0.5
```

3 Generated Slurm allocation

When the Tcl driver is started outside an allocation and `queueSystem = slurm`, it generates and submits `d3d-mormerge_qsubgo.sh` using `sbatch`. The generated script now contains explicit Slurm resources:

```
#SBATCH --job-name=mormerge
#SBATCH --nodes=<nodes>
#SBATCH --ntasks-per-node=<slurmTasksPerNode>
#SBATCH --cpus-per-task=<slurmCpusPerTask>
#SBATCH --time=<timeLimit>
#SBATCH --partition=<queue>
#SBATCH --output=d3d-mormerge-%j.out
#SBATCH --error=d3d-mormerge-%j.err
```

It also exports common thread-count variables from `SLURM_CPUS_PER_TASK`, disables Slurm CPU binding with `SLURM_CPU_BIND=none`, prints basic job metadata, expands `SLURM_JOB_NODELIST` with `scontrol show hostnames`, and then re-enters `mormerge.tcl` with the internal `-n` node count.

4 Node mapping

Inside the allocation, `mormerge.tcl` expands the Slurm node list using `scontrol show hostnames "$SLURM_JOB_NODELIST"`. It then builds a stable mapping:

```
required nodes =
    slurmControlNodes
    + number_of_conditions * slurmNodesPerCondition
```

If `slurmNodesPerCondition = 0`, the condition nodes are derived automatically by evenly dividing the non-control nodes over the conditions. If the division is ambiguous, the script fails early and asks for an explicit `slurmNodesPerCondition` value. The legacy one-node Slurm case is still allowed; all conditions share that node and the script prints a warning.

5 Launch model

The launch order remains unchanged:

- 1 clone `input` into one condition directory per condition;
- 2 clear `merge/sync`;
- 3 start one MorMerge helper process per run id;
- 4 wait for each `merge<runid>` sync marker;
- 5 start one model process per condition;
- 6 wait for condition sync markers and for launched processes to finish.

For Slurm, the MorMerge helper scripts are run on the batch/control node as ordinary serial processes. This avoids turning the helper into an MPI job step and preserves the existing stream-file protocol.

For DIMR conditions, `dimrexedir` now resolves to the direct `dimr` executable instead of `run_dimr.sh`. The condition launcher performs directory setup once, then starts DIMR with an explicit job step:

```
srun --mpi=<slurmMPI> \  
    --nodes=<slurmNodesPerCondition> \  
    --ntasks=<slurmNodesPerCondition * slurmTasksPerNode> \  
    --ntasks-per-node=<slurmTasksPerNode> \  
    --cpus-per-task=<slurmCpusPerTask> \  
    --odelist=<condition-node-list> \  
    "<dimrexedir>/dimr" <dimrargs>
```

Direct Delft3D-FLOW execution through `flowexedir` is intentionally limited to one Slurm task per condition, because launching a serial FLOW executable with multiple `srun` tasks would start multiple copies. Use `dimrexedir` for MPI-partitioned DIMR runs. Custom `runscript` entries are not wrapped in `srun`; if a custom run script needs MPI, it must contain its own site-specific launch command.

6 Wrapper change

`run_mormerge.sh` now forwards `--NNODES <nodes>` to `mormerge.tcl` as the internal `-n <nodes>` argument. This makes the wrapper usable when a site script has already created the Slurm allocation and only wants the Tcl driver to use the existing node count.

7 Operational notes

Set `slurmControlNodes = 1` for production multi-node MPI runs when the allocation can spare a control node. Set it to 0 only when MorMerge helper processes may share the first condition node. Keep `slurmTasksPerNode` aligned with the MPI decomposition in the DIMR configuration, and keep `slurmCpusPerTask` aligned with the intended OpenMP or threaded-library count per rank.

The generated Slurm scripts are retained when `debug = 1` in the `.mm` file. That is the recommended mode for first runs on a new cluster, because it leaves the generated `sbatch`, condition launcher, and MorMerge launcher scripts available for inspection.